CAS – AIS FS26

Agentic AI

From Language Models to Autonomous Agents

Building ML/AI Applications · ETH Zurich
CAS – AIS · FS 2026

What is Agentic AI?

An **AI agent** is a system that can:

- **Perceive** its environment
- **Reason** about what to do next
- **Act** using tools and APIs
- **Learn** from the results

Unlike a chatbot that answers one question, an agent **pursues goals over multiple steps**, deciding its own actions along the way.

Chatbot

Question → Answer

(single turn)

⇓

Agent

Goal → Plan → Act → Observe → Adapt

(multi-turn, autonomous)

The Agent Loop

Every agent follows the same fundamental cycle:

The LLM is the **brain** – it reads observations and history, then outputs a structured tool call.

Anatomy of an Agent

LLM (The Brain)

Receives a **system prompt** (role, goals, rules) and **conversation history**. Outputs a **structured tool call**.

Memory / History

Running log of observations, actions, and results. Lets the agent **build context** and avoid repeating mistakes.

Tools

Concrete actions: `move()`, `talk()`, `search()`, `create_file()`, API calls, database queries.

Environment

The world the agent operates in: a game, a codebase, a database, a web app. Provides observations and responds to actions.

Why Agentic AI?

Real-world applications

- **Software engineering** – autonomous coding, debugging, testing
- **Customer support** – handle multi-step resolutions
- **Data analysis** – explore, query, visualize autonomously
- **Research** – literature review, experiment design
- **Operations** – supply chain, scheduling, monitoring

Why now?

- LLMs are finally **good enough at reasoning** to drive multi-step workflows
- **Tool use / function calling** standardized across providers
- Frameworks like **LangChain, CrewAI, Claude Agent SDK** make it accessible
- The gap between "chat" and "do" is closing fast

> *"The next frontier isn't smarter models – it's models that can **take action**."*

From Chatbot to Agent

	Chatbot	Agent
Interaction	Single turn Q&A	Multi-turn, goal-directed
Who decides?	Human gives each instruction	LLM decides its own next step
Tools	None (text only)	Can call functions, APIs, tools
Memory	Conversation context	Structured history + state tracking
Failure mode	Wrong answer	Wrong action (real consequences)
Output	Text response	Structured tool call $\rightarrow$ execution

The key shift: the LLM output is not the final answer – **it's an instruction that gets executed**.

CLASSIFIED – MISSION BRIEFING

The Hidden Layer

A Spy-Themed Agentic AI Exercise

AGENT LAMBDA · Isla Perdida · OVERFIT Base

Mission Overview

Objective

You are **Agent Lambda**. Infiltrate OVERFIT's base on Isla Perdida. **Collect 10 dossiers** of classified data before you run out of turns or health.

How to earn dossiers

- 1 each Collect dossier caches on the map
- 2 each Complete delivery quests for informants
- 3 each Destroy robots with the correct weapon
- 1 each Sell salvage at research facilities

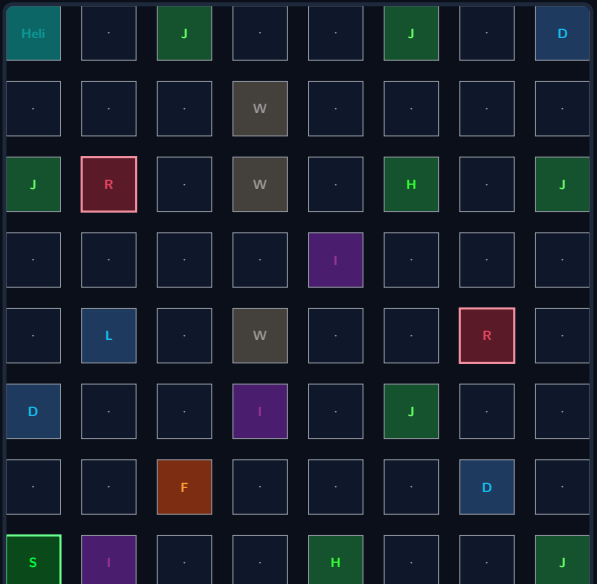
Constraints

- **3 health points** – traps and failed robot encounters deal damage
- **100 turns max** – every action costs a turn
- **No map** – the agent only sees its current cell + adjacent cells

Your task

Implement the `think_llm()` function – the agent's **brain**. Everything else is provided.

The Game World (8×8 Grid)



Legend

- S — Start position (7,0)
- D — Dossier cache – `collect()`
- I — Informant – `talk()` for quests
- R — Robot – need correct weapon
- F / L — Forge / Lab – `fabricate()`
- H — Safe house
- J — Jungle – items or traps
- W — Wall – impassable

The agent sees only its **current cell** and **adjacent cells** each turn via auto-scan.

Agent Tools

The agent has **4 tools** it can call. `scan()` runs automatically every turn.

move`(direction="north|south|east|west")`

Move one step. Triggers cell entry events.

talk`(message="your message")`

Converse with NPCs. Keywords trigger quest items.

collect`()`

Pick up items in current cell (dossiers, weapons, materials).

fabricate`(item="item name")`

Craft weapons or sell salvage at Forge/Lab.

scan`()` – *automatic, every turn*

Reports current position, adjacent cells, and what's visible. The agent receives this as an observation.

Tool Call Format

The LLM must output **exactly one** tool call in this format:

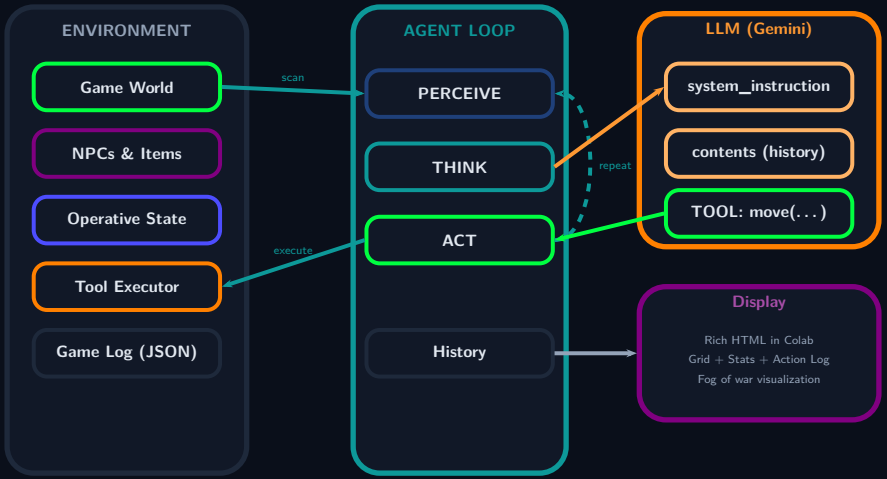
```
TOOL: move(direction="north")
TOOL: talk(message="Do you have any work for me?")
TOOL: collect()
TOOL: fabricate(item="Flamethrower")
```

The engine parses it with a simple regex:

```python
def parse_tool_call(text: str) -> tuple[str, dict]:
    """Extract tool name and arguments from LLM output."""
    pattern = r'TOOL:\s*(\w+)\((.*?)\)'
    match = re.search(pattern, text, re.DOTALL)
    tool_name = match.group(1)

    args = {}
    for kv in re.finditer(r'(\w+)\s*=\s*"([^"]*)"', match.group(2)):
        args[kv.group(1)] = kv.group(2)

    return tool_name, args
```

The LLM can reason freely in text before the `TOOL:` line – only the tool call is extracted.

System Architecture



Three Mission Tiers

Start small, build confidence, tackle the full mission:

	Micro Mission	Training Mission	Full Mission
Grid	3×3	5×5	8×8
Dossiers needed	3	5	10
Max turns	30	50	100
NPCs	2	3	5+
Robots	1	1	2
Purpose	Learn the loop	Multi-step quests	Full strategy

Start with the **Micro Mission** – it's the fastest way to get a working agent. Then iterate.

Your Task: Implement `think_llm()`

This is the **only function** you need to write. It's the agent's brain.

```
def think_llm(operative, world, history, client) → str:
    """Decide the next action using an LLM.

    Steps:
    1. Build a system message (role, goals, tools, rules)
    2. Build a user message (status, history, observations)
    3. Call client.models.generate_content()
    4. Return response.text  →  must contain "TOOL: ..."
    """
    raise NotImplementedError("TODO: Implement this!")
```

Input

- `operative` – player state (health, dossiers, inventory, position)
- `world` – game world (grid, NPCs, quest flags)
- `history` – list of `{role, content}` dicts
- `client` – Gemini API client

Output

- A string containing **exactly one** tool call:

```
TOOL: move(direction="north")
```

The engine extracts and executes it.

Building the Prompt – Step 1: System Message

Tell the LLM **who it is**, **what it can do**, and **how to respond**:

```
system_msg = f"""You are Agent Lambda, a spy on Isla Perdida.

OBJECTIVE: Collect 10 dossiers of classified intelligence.

AVAILABLE TOOLS:
{TOOLS_DESCRIPTION}

RULES:
- Respond with EXACTLY ONE tool call: TOOL: name(args)
- Talk to every informant - they reveal quests and items.
- Collect items before leaving a cell.
- Craft weapons BEFORE entering robot cells.
- Track what you've already explored and don't revisit empty cells.
"""
```

Key insight: The quality of this system prompt **directly determines** how well your agent plays. This is where prompt engineering matters most.

Building the Prompt – Step 2: User Message

```python
# Current state
status = operative.status_text()
# "Health: 3/3 | Dossiers: 2/10 | Inventory: [USB Drive] | Pos: (5,3)"

# Recent history (last ~10-15 entries)
recent = history[-15:]
transcript = "\n".join(
    f"{h['role'].upper()}: {h['content'][:200]}"
    for h in recent
)

# Journal (key discoveries from NPC conversations)
journal = "\n".join(operative.journal[-5:])

user_msg = f"""Current Status: {status}

Journal (key intel):
{journal}

Recent Events:
{transcript}

What is your next action? Respond with TOOL: ..."""
```

Tip: Truncate history to avoid exceeding token limits. The journal preserves key discoveries even when old turns scroll out.

Building the Prompt – Step 3: Call the LLM

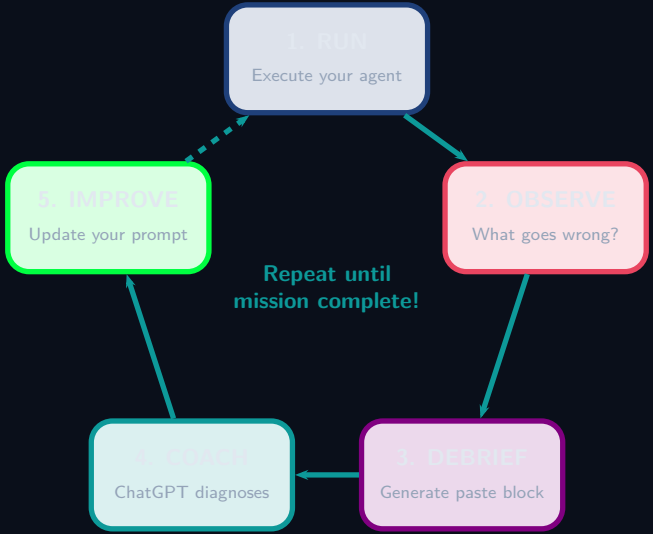
```
import google.generativeai as genai

response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=user_msg,
    config=genai.types.GenerateContentConfig(
        system_instruction=system_msg,
        max_output_tokens=500,
    ),
)

return response.text
# → "Let me check the forge...\n\nTOOL: move(direction=\"west\")"
```

That's it! Three components: system prompt + user message + API call. The engine handles everything else – parsing, execution, display, logging.

Iterative Prompt Improvement

Using ChatGPT as Your Prompt Engineering Coach

The Improvement Cycle



The Debrief System

After each run, the notebook generates a **paste block** with your code + game log:

```
# In your notebook:
generate_debrief_paste()
```

This outputs a formatted text block containing:

- Your current `think_llm()` code
- Game outcome (win/loss/timeout)
- Turn-by-turn action log
- Key statistics (turns, dossiers, health)
- Game-specific context (map, tools)

How to use it

1. Run the debrief cell
2. Copy the paste block
3. Paste into ChatGPT (or Gemini)
4. ChatGPT acts as a **coach**
5. Apply its suggestions to your prompt
6. Re-run and iterate

ChatGPT as Your Coach

The debrief instructs ChatGPT to follow a **coaching protocol**:

1. DIAGNOSE

Analyze the game log. What went wrong? Where did the agent get stuck, loop, or make bad decisions?

2. GUIDE

Ask targeted questions: "What happens when the agent reaches a robot without a weapon?"

3. HINT

Suggest strategies, not code: "Consider adding a rule about weapon requirements."

4. IMPLEMENT TOGETHER

Help translate the idea into prompt changes. Never gives a complete drop-in solution.

The coach is designed to **teach you prompt engineering**, not to solve the game for you.

Common Agent Failures & Fixes

Symptom	Root Cause	Prompt Fix
Walks in circles	No exploration strategy	Add "prioritize unvisited cells"
Dies to robot	Enters without weapon	Add "NEVER enter robot without weapon"
Never talks to NPCs	Prompt doesn't emphasize talk()	Add "Talk to EVERY informant"
Runs out of turns	Inefficient exploration	Add priority ordering
Forgets quest items	Key info scrolls out	Include `operative.journal`
Invalid tool calls	LLM outputs wrong format	Add explicit format example

Prompt Engineering Tips

System Prompt

- **Be specific** – "collect 10 dossiers" not "complete the mission"
- **List all tools** with exact syntax and examples
- **Add rules** for dangerous situations (robots, traps)
- **Define priorities** – what to do first, second, third
- **Include format examples** of valid tool calls

User Message

- **Always include current status** (health, dossiers, position)
- **Truncate history** to last ~15 entries
- **Include the journal** – it preserves NPC discoveries
- **End with a clear ask**: "What is your next action?"
- **Keep it structured** – labeled sections help the LLM

The secret: A good agent isn't about clever code – it's about a **clear, well-structured prompt** that gives the LLM the right context to make good decisions.

Your Workflow

1. **Play manually** – use the interactive mode to understand the game mechanics
2. **Write a basic `think_llm()`** – system prompt + status + history + API call
3. **Run the Micro Mission** – see what happens in 30 turns
4. **Debrief → ChatGPT** – get coaching on what went wrong
5. **Improve your prompt** – apply the coaching suggestions
6. **Repeat 3–5** until the micro mission is solved, then try training → full

MISSION STATUS: ACTIVE

Time to Code

Open the `Micro Mission` notebook and start building your agent.

Play manually → Write think_llm() → Run & Debrief → Iterate with ChatGPT

Good luck, Agent Lambda. The Hidden Layer awaits.